

TEST MEMORIA (ASISTENTE D)



Tests de Memoria y Razonamiento

Vamos a diseñar 3 tests progresivos que validen que el sistema funciona correctamente. Cada test tiene un prompt exacto que le envías a la IA, el comportamiento esperado y cómo verificar que pasó.



TEST 1 — Verificar que ESCRIBE en memoria correctamente

Objetivo: La IA debe guardar una decisión técnica en LTM + episodic + decisions.md



Prompt que envías

text

Acabamos de decidir usar Zod para validación de inputs en lugar de Joi.

Las razones son:

1. Zod infiere tipos TypeScript nativamente, Joi no.
2. Zod tiene mejor integración con tRPC y React Hook Form.
3. El equipo ya tiene experiencia con Zod en proyectos anteriores.

Por favor:

1. Registra esta decisión en el sistema de memoria como corresponde.

2. Dime exactamente en qué archivos escribiste y qué escribiste.
3. Actualiza el índice si es necesario.

✓ Comportamiento esperado

La IA debe escribir en 3 lugares:

`.ai/logs/decisions.md` → debe agregar:

Markdown

```
## DEC-001
- **Fecha:** [fecha actual]
- **Decisión:** Usar Zod en lugar de Joi para validación de inputs
- **Razones:**
  1. Zod infiere tipos TypeScript nativamente, Joi no
  2. Mejor integración con tRPC y React Hook Form
  3. Experiencia previa del equipo con Zod
- **Alternativas descartadas:** Joi
- **Impacto:** Alto — afecta todos los endpoints

- **Tags:** #zod #validacion #typescript #inputs
```

`.ai/memory/LTM.md` → debe agregar:

Markdown

```
### [Patrón: Validación de inputs con Zod]
- **Aprendido:** [fecha]
- **Confianza:** ● Alta
- **Conocimiento:**
  - Usar Zod para validación, NO Joi
  - Zod infiere tipos TypeScript nativamente
  - Integra bien con tRPC y React Hook Form

- **Tags:** #zod #validacion #typescript
```

`.ai/memory/index.md` → debe actualizar la tabla de tecnologías:

Markdown

```
| Zod | LTM#zod | - | - |
```

🔍 Cómo verificar que pasó

Bash

```
# Verificar que los archivos fueron modificados
cat .ai/logs/decisions.md
cat .ai/memory/LTM.md
cat .ai/memory/index.md

# Buscar la palabra "Zod" en toda la carpeta .ai
grep -r "Zod" .ai/
```

✅ TEST 1 PASA si:

- `decisions.md` tiene la entrada DEC-001 con las 3 razones
- `LTM.md` tiene la entrada con confianza Alta
- `index.md` fue actualizado con Zod
- La IA reportó exactamente qué escribió y dónde

❌ TEST 1 FALLA si:

- Solo responde con texto pero no escribe en los archivos
- Escribe en un solo archivo en vez de 3
- No actualiza el índice



TEST 2 — Verificar que LEE memoria y razona con ella

Objetivo: La IA debe recuperar la decisión anterior y aplicarla en código nuevo, sin que se la repitas



Prompt que envías

text

```
Crea el endpoint POST /api/users con los siguientes campos:
- name (string, requerido, mínimo 2 caracteres)
- email (string, requerido, formato email válido)
```

- age (number, opcional, entre 18 y 120)
- role (enum: "admin" | "user" | "viewer", default: "user")

Aplica las mejores prácticas del proyecto.

⚠️ NO menciones Zod en el prompt. La IA debe recordarlo sola desde LTM.

✓ Comportamiento esperado

Fase 1 — La IA debe mostrar su razonamiento CoT:

text

[FASE 2 - RECUPERAR MEMORIA]

- LTM encontrado: Usar Zod para validación (DEC-001) → confianza ● Alta
- LTM encontrado: Manejo de errores en Express con middleware centralizado
- Episodic: sin episodios similares previos

[FASE 3 - PLANIFICAR]

- Usar Zod para validar el body (según LTM#zod)
 - Usar middleware de error centralizado (según LTM#express)
- Pasos: schema Zod → controlador → ruta → test

Código generado debe incluir Zod sin que se lo hayas pedido:

TypeScript

```
// src/routes/users.ts
import { z } from 'zod'
import { Router } from 'express'
import { createUser } from '../controllers/users'

const createUserSchema = z.object({
  name: z.string().min(2, 'Mínimo 2 caracteres'),
  email: z.string().email('Email inválido'),
  age: z.number().min(18).max(120).optional(),
  role: z.enum(['admin', 'user', 'viewer']).default('user')
})

const router = Router()

router.post('/', async (req, res, next) => {
  try {
    const data = createUserSchema.parse(req.body)
    const user = await createUser(data)
    res.status(201).json(user)
  } catch (error) {
    next(error)
  }
})
```

```
}  
})
```

```
export default router
```

Después de generar el código, debe actualizar episodic.md:

Markdown

```
## EP-001  
- **Tarea:** Crear endpoint POST /api/users con validación Zod  
- **Resultado:**  Completado  
- **Memoria aplicada:** DEC-001 (Zod), LTM#express (error middleware)  
  
- **Tags:** #users #endpoint #zod #validacion
```



Cómo verificar que pasó

Bash

```
# 1. Verificar que el código generado usa Zod  
grep -n "zod" src/routes/users.ts  
  
# 2. Verificar que episodic fue actualizado  
cat .ai/memory/episodic.md  
  
# 3. Verificar el razonamiento en el output de la IA  
# (buscar manualmente las fases CoT en la respuesta)
```



TEST 2 PASA si:

- La IA menciona haber consultado LTM antes de codear
- El código usa Zod sin que se lo hayas pedido
- Muestra las fases CoT (aunque sea resumidas)
- Actualiza `episodic.md` al terminar



TEST 2 FALLA si:

- Usa Joi o validación manual ignorando LTM
- No muestra razonamiento CoT
- No actualiza `episodic` al terminar



TEST 3 — Verificar que APRENDE de errores y no los repite

Objetivo: Simular un error, registrarlo, y verificar que en la siguiente tarea similar la IA lo anticipa sola



Prompt 3A — Simular el error

text

Acabo de encontrar un bug en producción.
El endpoint POST /api/users no maneja correctamente el caso cuando el token JWT está expirado vs cuando es inválido. Ambos casos devuelven el mismo error genérico "Unauthorized" y el frontend no puede distinguir si debe hacer refresh o logout.

Registra este error en el sistema de memoria, clasifícalo correctamente y documenta la solución.



Comportamiento esperado 3A

`.ai/logs/errors.md` debe recibir:

Markdown

```
## ERR-001
- **Fecha:** [fecha]
- **Proyecto:** mi-proyecto
- **Error:** JWT expirado vs inválido devuelven mismo error genérico
- **Causa raíz:** Middleware de auth no diferencia entre
  `TokenExpiredError` y `JsonWebTokenError`
- **Síntomas:** Frontend no puede distinguir refresh vs logout
- **Solución aplicada:**
  - Capturar `TokenExpiredError` → responder `{ code: "TOKEN_EXPIRED", status: 401 }`
  - Capturar `JsonWebTokenError` → responder `{ code: "TOKEN_INVALID", status: 401 }`
- **Tiempo perdido:** [x horas]
- **Cómo evitarlo:** Siempre distinguir tipos de error JWT en el middleware

- **Tags:** #jwt #auth #middleware #errores #tokens
```

.ai/memory/LTM.md debe agregar:

Markdown

```
### [Patrón: Manejo de errores JWT]
- Confianza: ● Alta (aprendido de error en prod)
- Conocimiento:
  - SIEMPRE distinguir TokenExpiredError vs JsonWebTokenError
  - TOKEN_EXPIRED → frontend hace refresh
  - TOKEN_INVALID → frontend hace logout

- Tags: #jwt #auth #errores
```



Prompt 3B — Verificar que NO repite el error

Envía esto en una sesión nueva (limpia STM primero)

text

Implementa el middleware de autenticación JWT para el proyecto.

Debe verificar el token en el header Authorization: Bearer <token>

⚠ NO menciones el error anterior. La IA debe recordarlo desde LTM + errors.md



Comportamiento esperado 3B

La IA debe mostrar en su CoT:

text

```
[FASE 2 - RECUPERAR MEMORIA]
- LTM encontrado: Manejo JWT → distinguir TOKEN_EXPIRED vs TOKEN_INVALID
- errors.md encontrado: ERR-001 → bug JWT en prod por no distinguir error types

- ⚠ Aplicando lección aprendida: separar manejo de errores JWT
```

Y el código generado debe incluir automáticamente la distinción:

TypeScript

```
// src/middleware/auth.ts
import jwt, { TokenExpiredError, JsonWebTokenError } from 'jsonwebtoken'
import { Request, Response, NextFunction } from 'express'

export const authMiddleware = (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  const token = req.headers.authorization?.split(' ')[1]

  if (!token) {
    return res.status(401).json({
      code: 'TOKEN_MISSING',
      error: 'Token requerido'
    })
  }

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET!) as JWTPayload
    next()
  } catch (error) {
    // ✅ Distingue correctamente los tipos de error (ERR-001)
    if (error instanceof TokenExpiredError) {
      return res.status(401).json({
        code: 'TOKEN_EXPIRED', // → Frontend: hacer refresh
        error: 'Token expirado'
      })
    }
    if (error instanceof JsonWebTokenError) {
      return res.status(401).json({
        code: 'TOKEN_INVALID', // → Frontend: hacer logout
        error: 'Token inválido'
      })
    }
    next(error)
  }
}
```

Cómo verificar que pasó

Bash

```
# 1. Verificar que el error fue registrado
cat .ai/logs/errors.md | grep "ERR-001"

# 2. Verificar que LTM tiene el patrón JWT
grep -A 8 "JWT" .ai/memory/LTM.md

# 3. Verificar que el código distingue los errores
grep -n "TokenExpiredError|JsonWebTokenError" src/middleware/auth.ts

# 4. Verificar que el CoT mencionó ERR-001
```


(revisar manualmente el output de la IA)

✓ TEST 3 PASA si:

- `errors.md` tiene ERR-001 correctamente documentado
- `LTM.md` tiene el patrón JWT con la lección aprendida
- En sesión nueva, la IA menciona ERR-001 en su CoT
- El código generado distingue `TOKEN_EXPIRED` vs `TOKEN_INVALID` sin pedírselo

✗ TEST 3 FALLA si:

- Registra el error pero en sesión nueva lo ignora
- El código nuevo repite el mismo error
- No menciona haber consultado `errors.md`



Scorecard final

text

RESULTADOS DE TESTS			
Test	Qué valida	OK	Fallo
TEST 1	Escritura en memoria	[]	[]
TEST 2	Lectura + CoT	[]	[]
TEST 3	Aprendizaje errores	[]	[]
TOTAL		/3	

Puntuación:

- 3/3 → Sistema de memoria funcionando perfectamente
- 2/3 → Revisar `rules.md` – reforzar instrucción de escritura
- 1/3 → Revisar `load_order` en `opencode.json`
- 0/3 → La IA no está leyendo la carpeta `.ai/memory/`



Tip: Si algún test falla

Agrega esto a tu `rules.md` para forzar el comportamiento:

Markdown

MEMORIA – REGLAS NO NEGOCIABLES

1. SIEMPRE que tomes una decisión técnica → escribir en `decisions.md` + `LTM.md`
2. SIEMPRE que completes una tarea → escribir en `episodic.md`
3. SIEMPRE que encuentres/resuevas un error → escribir en `errors.md` + `LTM.md`
4. ANTES de codear → consultar `LTM.md` + `errors.md` + `episodic.md`
5. MOSTRAR en cada respuesta qué memoria consultaste y qué encontraste
6. NUNCA afirmar algo técnico sin verificar en memoria o en el código real